Chalmers Challenge

2025

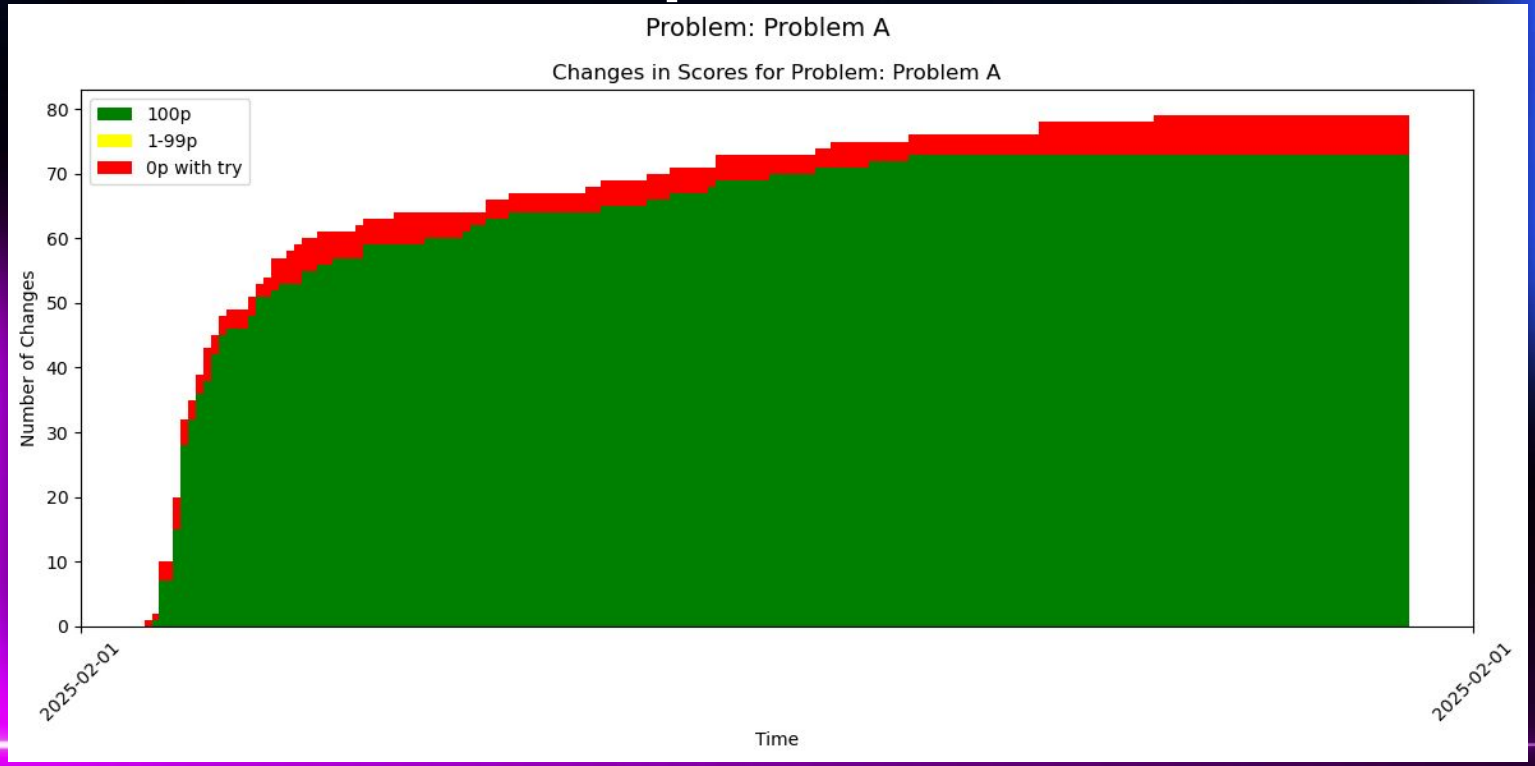
A - Cups and Balls

- Problem
  - Cups and balls
  - Keep track of which cup the ball is underneath to help figure out what cups to swap to make Hugo guess correctly
- Solution
  - Keep track of which cup the ball resides under using an integer between 1 and 3, re-assigning it when one of the cups is equal to it
  - Handle the case when the ball is under Hugo's guessed cup separately, and select the other cups
  - Else switch Hugo's cup with the cup that the ball is under

A - Cups and Balls

```
ball = 2
hugo_guess = int(input())
for i in range(int(input())):
    swap1, swap2 = map(int, input().split())
    if ball == swap1:
        ball = swap2
    elif ball == swap2:
        ball = swap1

if ball == hugo_guess:
    other_cups = "123".replace(str(ball), _new:"")
    print(other_cups[0], other_cups[1])
else:
    print(hugo_guess, ball)
```

A - Cups and Balls



B - Overnight Oats

- Problem
  - *N*-day period, X-day shelf life
  - Each evening, Julia can either do nothing (PASS), EAT or ADD overnights oats
  - In the end there will be no overnight oats left
  - Will Julia ever eat expired overnight oats?
- Solution
  - Simulate the events, add overnight oats when needed, remove overnight oats when needed

B - Overnight Oats

- Pseudo code:
  - For every day:
    - If PASS:
      - continue
    - Else if ADD:
      - Add to refrigerator
    - Else if EAT:
      - Remove *oldest* overnight oats
      - If expired, exit with “ono..”
  - print “yay!”

B - Overnight Oats

- Pseudo code:
  - For every day:
    - If PASS:
      - continue
    - Else if ADD:
      - Add to refrigerator
    - Else if EAT:
      - Remove *oldest* overnight oats
      - If expired, exit with “ono..”
  - print “yay!”

Quadratic solution if this step is linear (e.g. popping first element of a list/looping to find oldest)

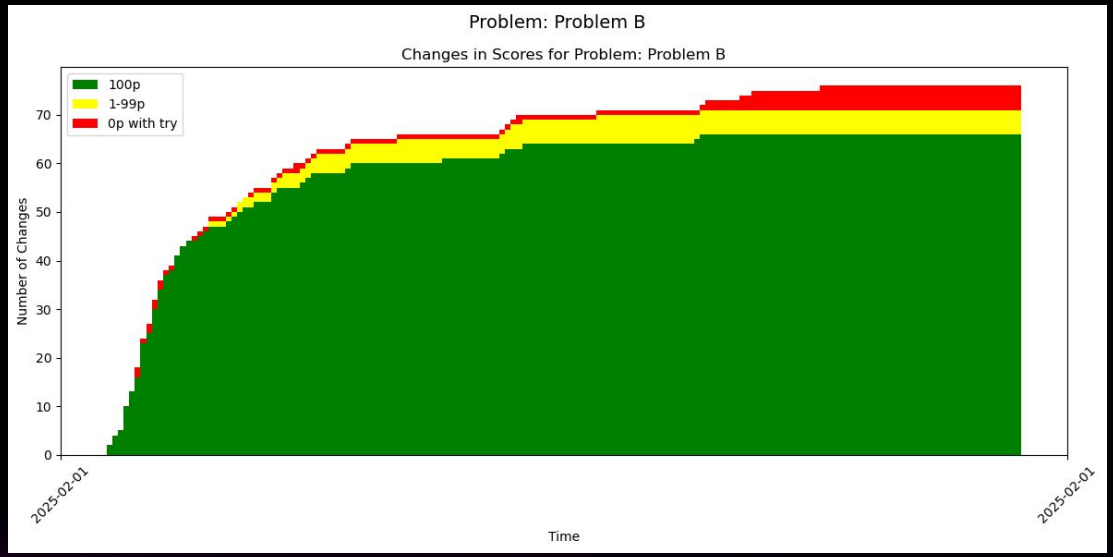
Linear solution if this is constant time (queue)

B - Overnight Oats

```python
import heapq


days = int(input())
shelf = int(input())


oatlist = []


for i in range(days):
    action = input()
    if action == "ADD":
        heapq.heappush(oatlist, i)
    if action == "PASS":
        continue
    if action == "EAT":
        cur_oat_birthday = heapq.heappop(oatlist)
        if i - cur_oat_birthday > shelf:
            print("ono..")
            exit()

print("yay!")
```

_BuyHighSellLow (Rickard)

B - Overnight Oats



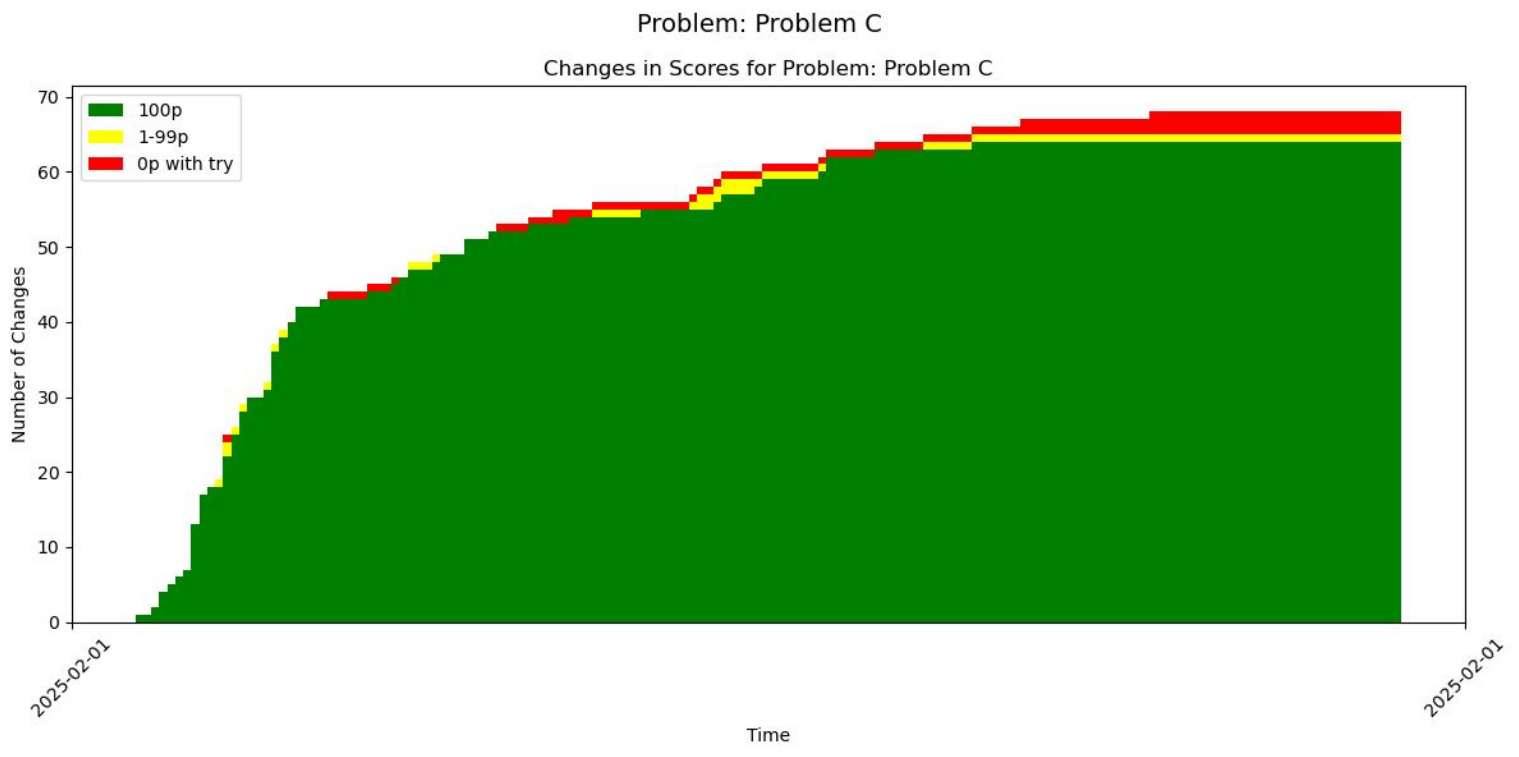
C - Passing Time

- Problem:
  - Some cards have been drawn from a shuffled deck of cards
  - You are tasked with finding the probability that you can correctly guess the top card's rank, using the most optimal strategy
- Solution:
  - The most optimal strategy will be to pick the rank that there are most cards of.
  - This means we can keep track of the counts of the different ranks using a dictionary
  - Then we can divide the maximum number of one of the ranks with the amount of cards remaining in the deck

C - Passing Time

```
ranks = ["A", "2", "3", "4", "5", "6", "7",
         "8", "9", "10", "J", "Q", "K"]
counts = {r:4 for r in ranks}

n = int(input())
for _ in range(n):
    card = input()
    rank = card[:-1]
    counts[rank] -= 1

most = max(counts.values())
remaining_cards = 52 - n
print(most / remaining_cards)
```

C - Passing Time



D - Bouquet

Sample Input 2

```
5
-1 0 B
-3 0 R
2 3 R
1 1 G
-3 3 G
```

Sample Output 2

```
8.537319187990756
```

D - Bouquet

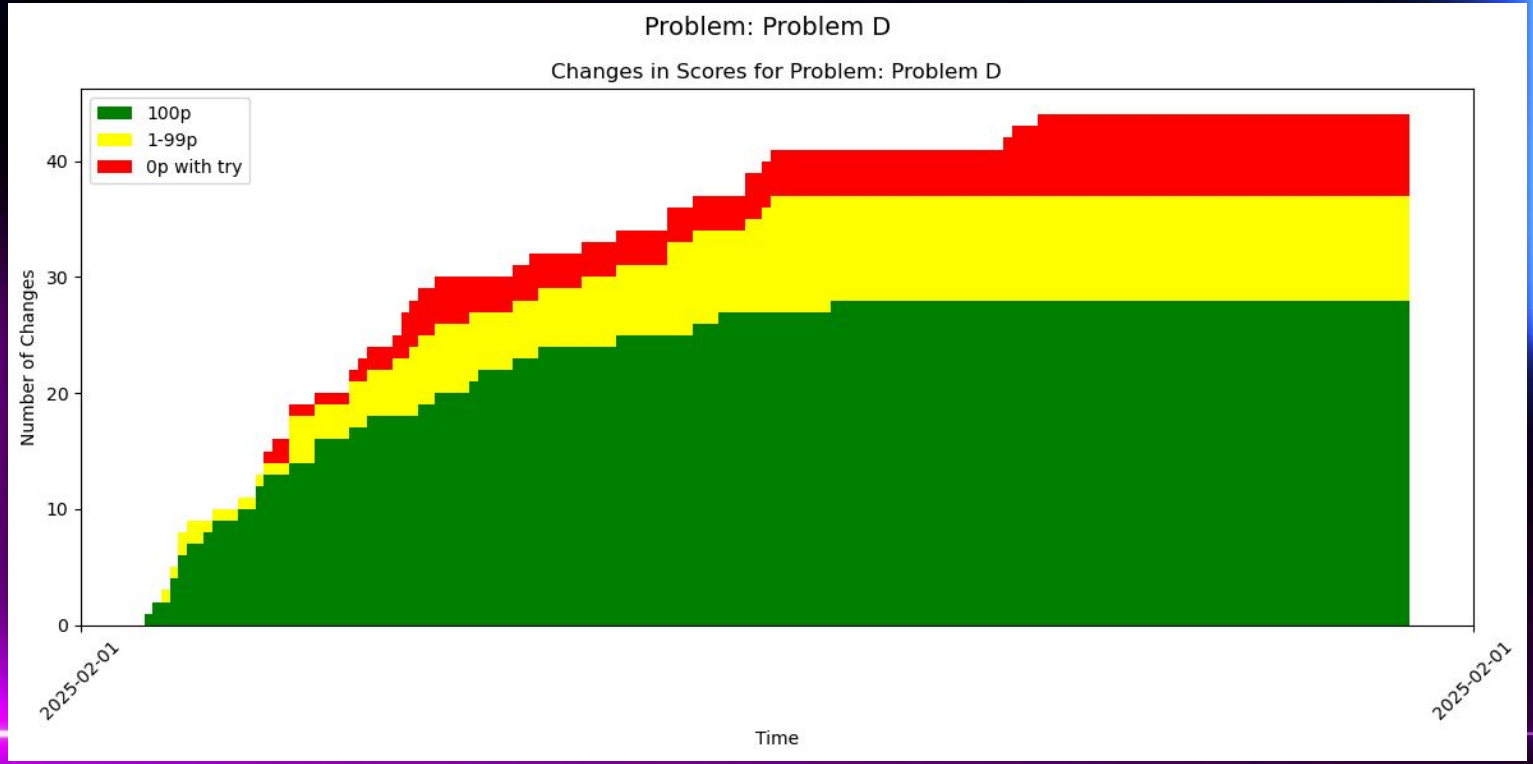
```python
n = int(input())
pos = []
colors = []
for i in range(n):
    x, y, color = input().split()
    pos.append((int(x), int(y)))
    colors.append(color)

ans = 1e10
for f1 in range(n):
    for f2 in range(n):
        for f3 in range(n):
            if len({colors[f1], colors[f2], colors[f3]}) == 3:
                ans = min(
                    ans,
                    dist((0,0), pos[f1]) + dist(pos[f1], pos[f2])
                    + dist(pos[f2], pos[f3]) + dist(pos[f3], (0, 0))
                )
print(ans)
```

D - Bouquet

```
double ans = 1e30;
fori(n) {
  forj(3) best[j] = 1e30;
  forj(n) if (c[j] != c[i]) {
    double d = sqrt(x[j] * x[j] + y[j] * y[j]) + sqrt((x[j] - x[i]) * (x[j] - x[i]) + (y[j] - y[i]) * (y[j] - y[i]));
    best[c[j]] = min(best[c[j]], d);
  }
  double cur = 0;
  forj(3) if (j != c[i]) cur += best[j];
  ans = min(ans, cur);
}
writeln(ans);
```

Alexander Kravberg

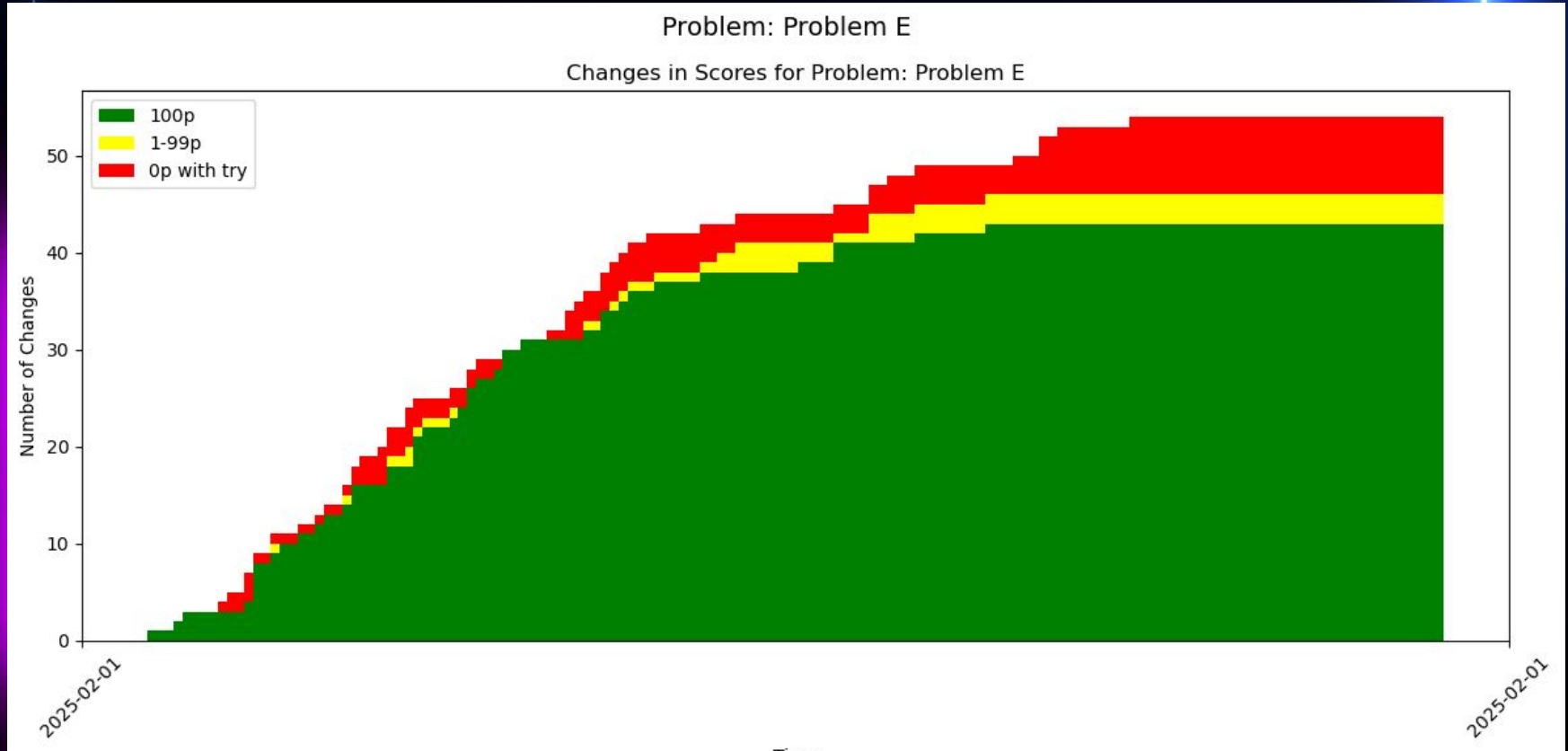
D - Bouquet


Problem: Problem D
Changes in Scores for Problem: Problem D
100p
1-99p
0p with try
Number of Changes
0
10
20
30
40
2025-02-01
2025-02-01
Time

E - Mountainpeeker

- Problem
  - How many mountain peaks can Joel see?
- Solution
  - 40 points: For each mountain peak, check if any other mountain peak is blocking the view, for example by comparing angles (or simply ratio (y-h) / x). O(n^2)
  - 50 points: Split mountains into left and right, they solve equivalently for both sides, setting x = -x for left side.
  - 100 points: Sort both sides in increasing x-distance, then go through the list and keep track of largest seen angle, every increase corresponds to a mountain peak in view. O(n log n)

E - Mountainpeeker



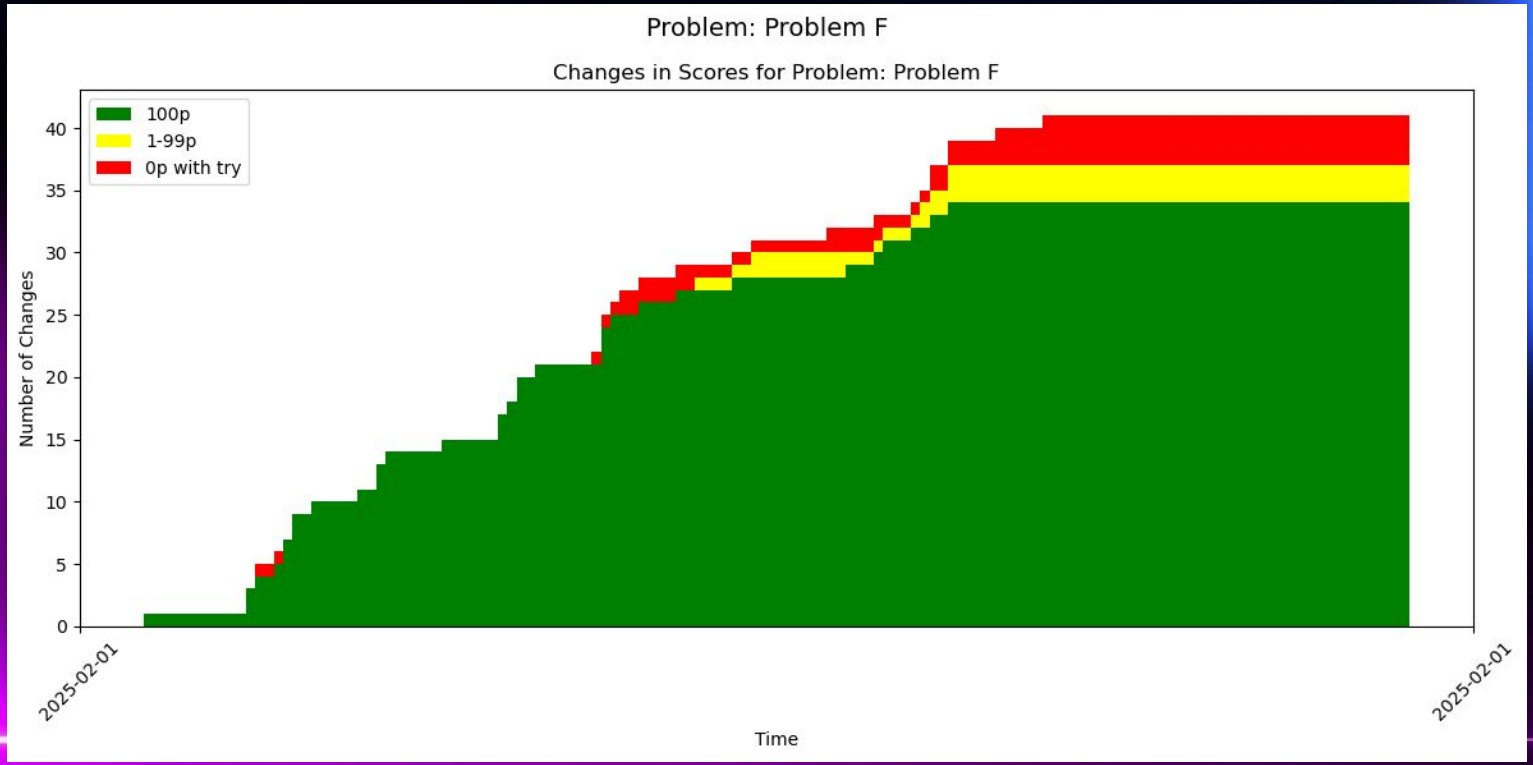
F - CCC Camp

- Problem
  - N points contributing 1 or 2 cups of coffee
  - M intervals
  - We want to count how many cups of coffee belonging to points that are not inside any interval
- Solution
  - Order points by their position
  - Order all intervals based on start position
  - Go through all the people one by one and keep a pointer for the current interval (starting at the first interval). If the current person is to the left of the current interval, we can mark it as not belonging to any interval, thus summing its cups of coffee, and if it is to the right we increase the interval pointer.

F - CCC Camp

```python
n, m = map(int, input().split())
a = []
for _ in range(n):
    name, pos = input().split()
    a.append((int(pos), 1 + (name in ("Joshua", "Gustav"))))
b = [tuple(map(int, input().split())) for _ in range(m)]
a.sort()
b.sort()
i = 0
cnt = 0
for p, c in a:
    while i < m and p > b[i][1]:
        i += 1
    if i >= m or b[i][0] > p:
        cnt += c
print(cnt)
```

F - CCC Camp


Problem: Problem F
Changes in Scores for Problem: Problem F
100p
1-99p
0p with try
Number of Changes
0
5
10
15
20
25
30
35
40
2025-02-01
2025-02-01
Time

G - Memento

There are multiple possible solutions.

Classic unlabeled graph isomorphism algorithms are exponential.

We basically need a property of graphs that is invariant (unchanging) to relabeling vertices. Perhaps there is a way to add the edges to achieve this.

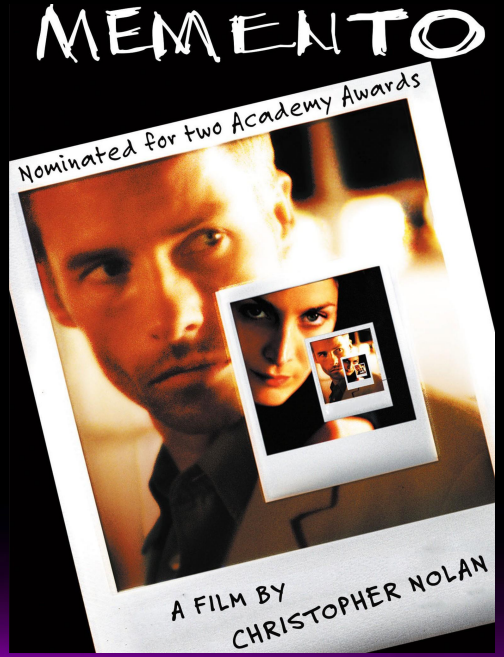
Consider adding the 30 edges to some arbitrary node. What is the probability that there will be another node with degree >= 30?

G - Memento

Remember that the graph is generated with maximum 4500 edges distributed uniformly to the 1000 vertices. The expected degree of a vertex is 4500*2/1000 = 9. You could calculate the variance here or probability of a vertex having a degree >= 30. Intuitively, the probability is very small. It turns out that it is negligible and the solution works.

You could be extra safe and instead of choosing an arbitrary node to add the 30 edges to, pick the node of highest degree to add the edges too and connect the edges to the vertices of lowest degrees.

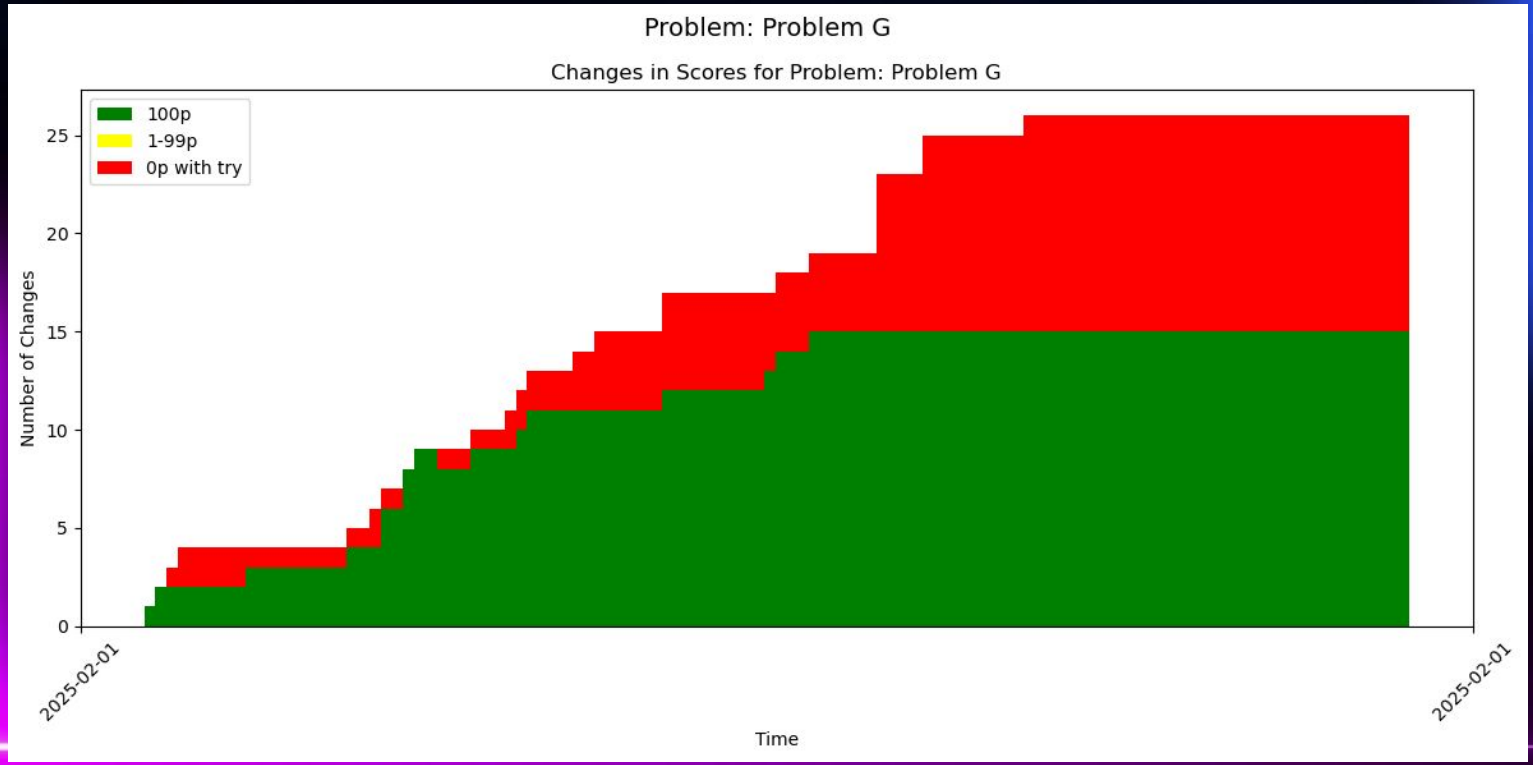
G - Memento


MEMENTO
Nominated for two Academy Awards
A FILM BY
CHRISTOPHER NOLAN

G - Memento

```python
def run(edges):
    degree = [0] * 1000
    for a, b in edges:
        degree[a] += 1
        degree[b] += 1
    if max(degree) >= 30:
        return []
    hasEdgeTo0 = [False] * 1000
    for a, b in edges:
        if a == 0:
            hasEdgeTo0[b] = True
        elif b == 0:
            hasEdgeTo0[a] = True
    ret = []
    for i in range(1, 1000):
        if not hasEdgeTo0[i]:
            ret.append([0, i])
            if len(ret) >= 30:
                break
    return ret

```

G - Memento


Problem: Problem G
Changes in Scores for Problem: Problem G
100p
1-99p
0p with try
Number of Changes
0
5
10
15
20
25
2025-02-01
2025-02-01
Time

H - Exploring Delsjön

There is a tree. We are given the distance between every pair of leaf nodes (nodes with degree 1) in it. Given this, find any tree which has the same distances between its leaves.

H - Exploring Delsjön

“Interior nodes” are non-leaf nodes. Add interior nodes to connect leaf 0 and 1. Then, add interior nodes to connect leaf 0,1,2. Etc.

Well-known fact: the distance between two nodes (a,b) in a tree is depth[a]+depth[b]-2*depth[lca(a,b)]

Let’s root the tree at leaf 0.

Now, we know the depth of all other leaves.

H - Exploring Delsjön

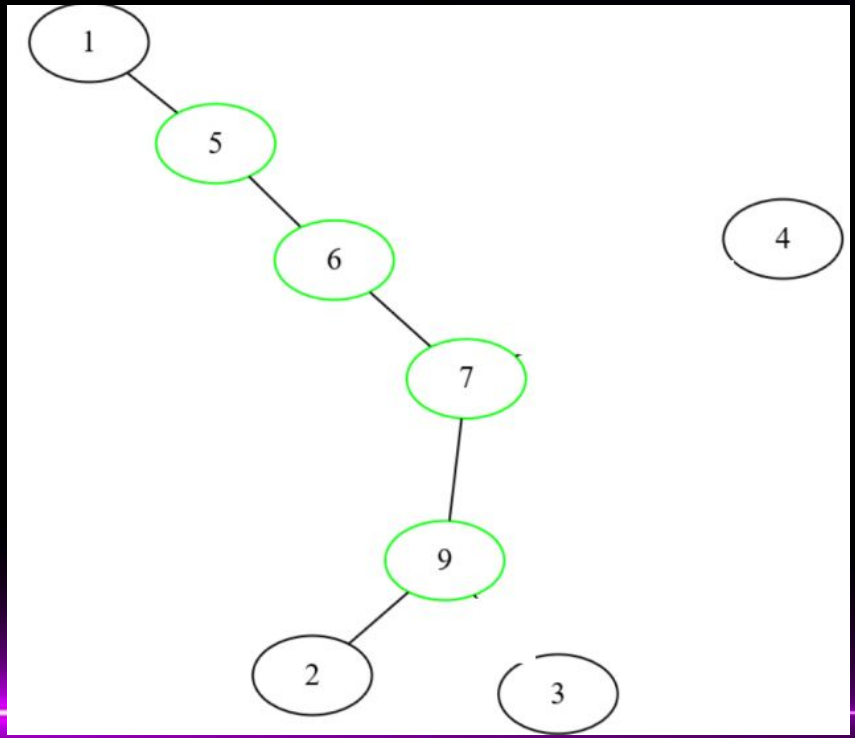
Since we know the depth of all leaves (0 is 0, rest is dist(0,a)), we can solve for the depth of the lca using

depth[a]+depth[b]-2*depth[lca(a,b)]

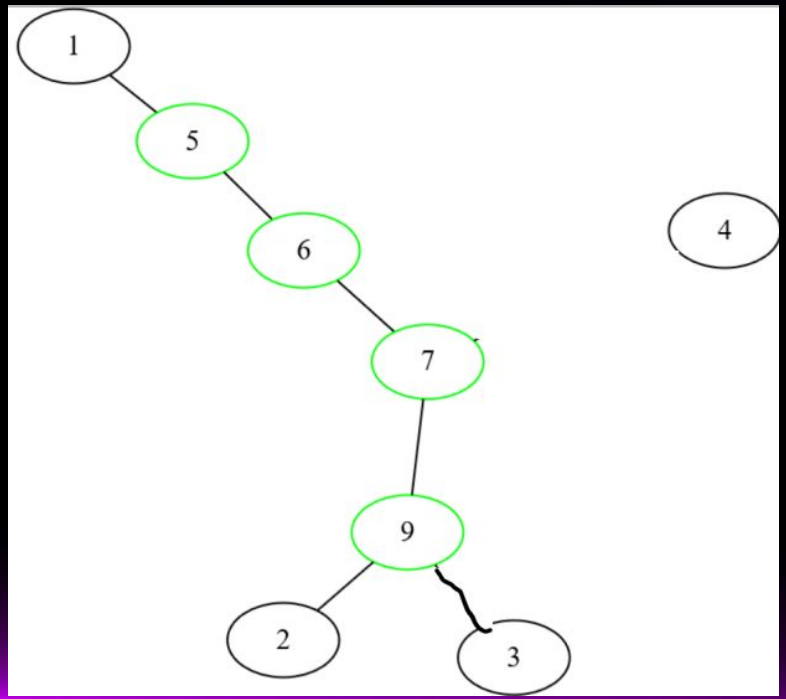
Let us try to insert leaf L. Compute the depth of lca(i,L) for each i < L. We will insert a path from L-> the deepest lca. Use difference in depth between L and the lca to find number of nodes

Implementation detail: add a path from leaf 0 to 1 instantly

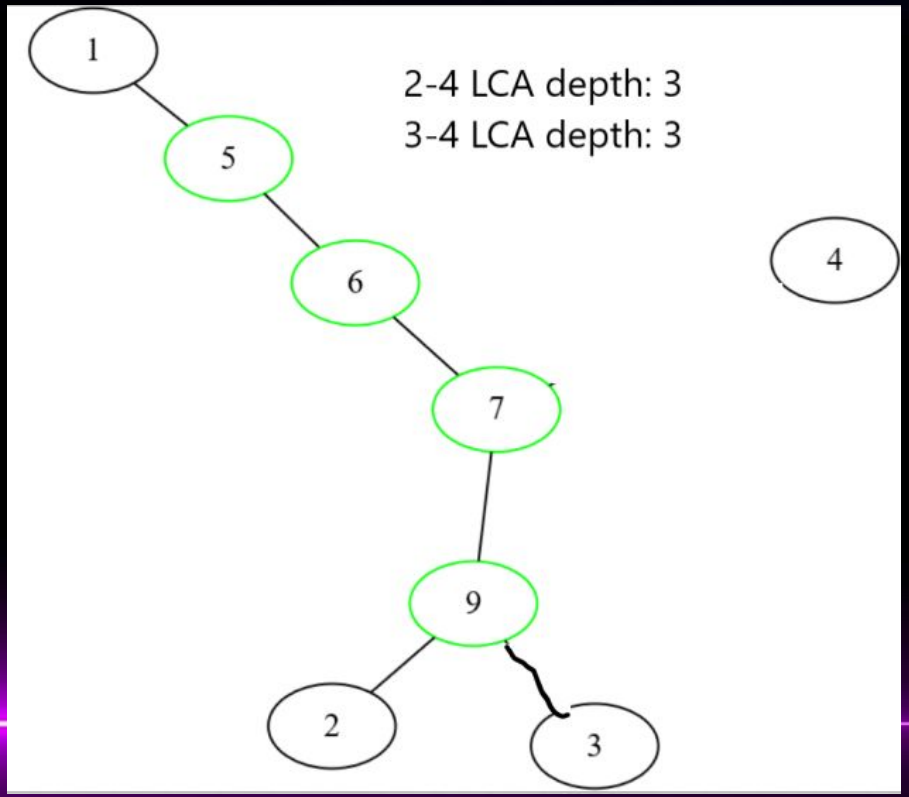
H - Exploring Delsjön



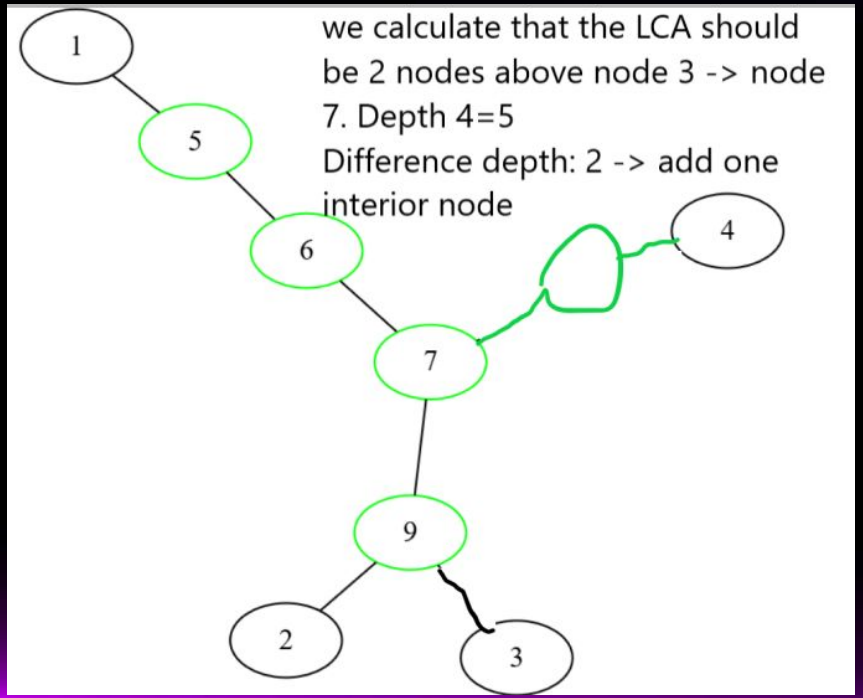
H - Exploring Delsjön



H - Exploring Delsjön



H - Exploring Delsjön



H - Exploring Delsjön

I - Polkagris

Problem: Can a given 1e7 long string be converted into another given string, using “folds”?

A fold is equivalent to removing the K first(last) characters of a 2K+1 long palindrome prefix(suffix)

```
ABABABCA
--ABABCA
---BABCA
----ABCA
```

I - Polkagris

Observation: We will only ever have substrings of the initial string.

Consider a graph with nodes corresponding to substring slices [L,R]. If s[L-1] = s[L+1] we have an edge from [L-1,R] to [L,R], etc.

$O(N^3)$: DP on this graph, then check all end states corresponding to substrings equal to the target.

I - Polkagris

```
N, T = [int(x) for x in input().split()]
initial = input()
target = input()

# whether initial can be folded into initial[l:r+1]
memo = {}
def dp(l,r):
    if l == 0 and r == len(initial)-1: return True
    if (l,r) in memo: return memo[l,r]
    k = r-l+1
    for i in range(1,k):
        assert l+i <= r
        if (l-i < 0) or initial[l+i] != initial[l-i]: break
        if dp(l-i,r):
            memo[l,r] = True
            return True
    for i in range(1,k):
        assert r-i >= l
        if (r+i >= len(initial)) or initial[r-i] != initial[r+i]: break
        if dp(l,r+i):
            memo[l,r] = True
            return True
    memo[l,r] = False
    return False

def possible(target):
    for l in range(len(initial)-len(target)+1):
        r = l+len(target)-1
        if initial[l:r+1] == target and dp(l,l+len(target)-1):
            return True
    return False

if possible(target) or possible(target[::-1]):
    print("possible")
else:
    print("impossible")
```

DP[L,R]

Recurse into DP[L-i,R] for 2i+1 long palindromes centered at L

Finally, at every offset into the string

1. Check if matches target
2. Check if DP is true

(Also check for target[::-1]!)

I - Polkagris

1st way of optimizing to quadratic:

Recurse into DP[L-i,R] for 2i+1 long palindromes centered at L
^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Can precompute length of palindrome centered at L in $O(N^2)$.

Can use prefix sum to check if *any* DP[L-i,R] for i in 1..(length of palindrome)

I - Polkagris

2nd way of optimizing to quadratic:

Folding one side is almost independent from the other side. Could we do 2 linear-state DPs instead of a single quadratic one?

Yes! Any folds large enough to affect the other side could WLOG be shortened by applying any of the other sides recent folds.

I - Polkagris

Applying both (prefix sum + two linear-state DPs) gives a linear solution, if

1. We compute palindrome widths in linear time (Manacher)
2. We find all substrings in the input equal to the target (KMP)

Both in KACTL

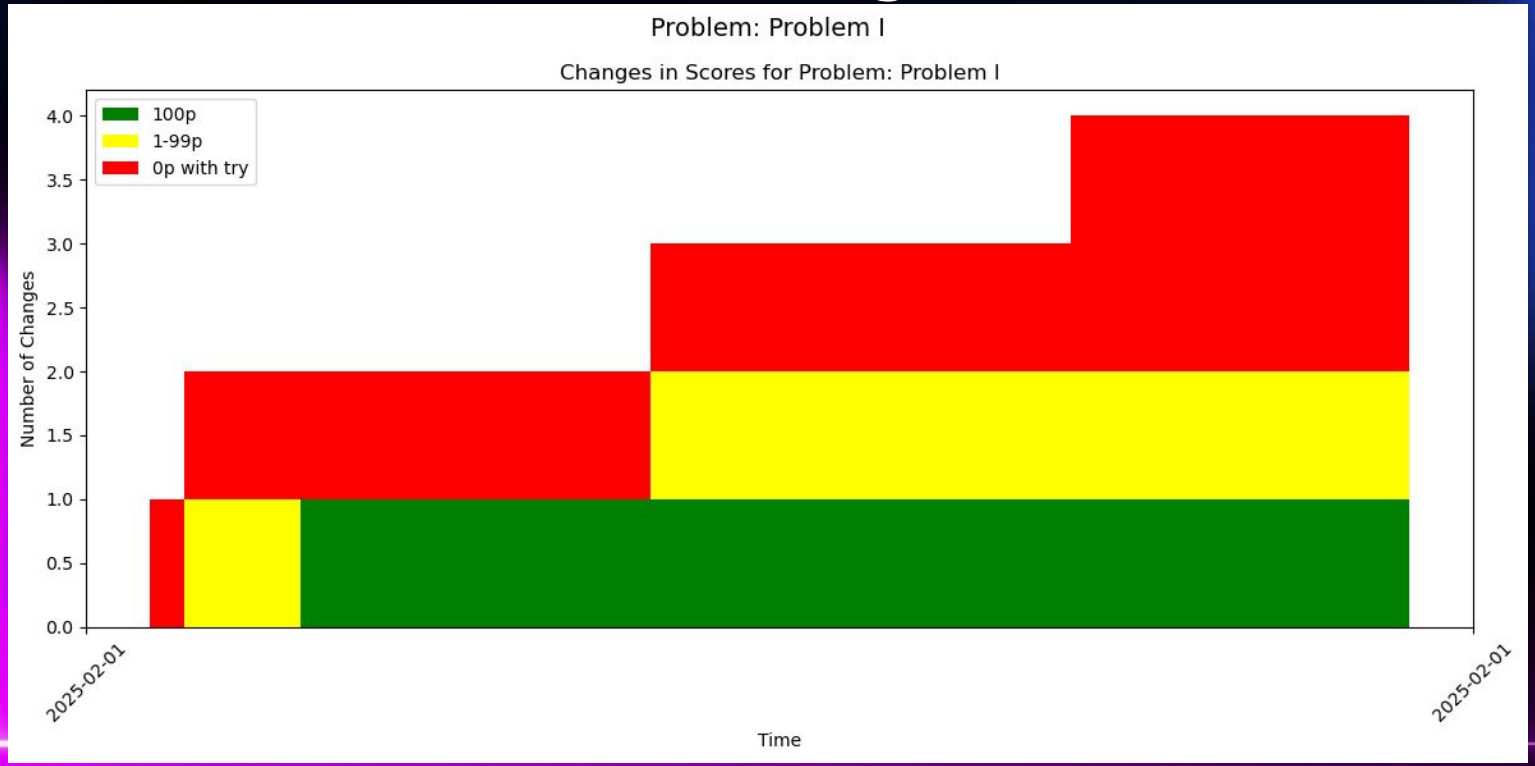
I - Polkagris

```cpp
int main(){
    cin.tie(0);
    ios::sync_with_stdio(false);

    ll n,m; cin>>n>>m;

    ll mxsz = n+3;
    pw.resize(mxsz);
    invpw.resize(mxsz);
    pw[0] = 1;
    invpw[0] = 1;
    rep(i,1,mxsz){
        pw[i] = pw[i-1]*base;
        invpw[i] = modInverse(pw[i]);
        // cout<<invpw[i]*pw[i]<<endl;
    }

    // cout<<modInverse(base)*base<<endl;



    string a; cin>>a;
    string b; cin>>b;

    IntervalHash ha(a);
    RevIntervalHash rha(a);

    // rep(i,0,9) cout<<ha.get(i,i+3)<<" ";
    // cout<<endl;
    // rep(i,0,9) cout<<rha.get(i,i+3)<<" ";
    // cout<<endl;

    IntervalHash hb(b);
    RevIntervalHash rhb(b);
    uint32_t bhs = hb.get(0,m);

    vector<bool> canCutPref(n);
    canCutPref[0] = true;
    ll closestCut = 0;
    rep(i,1,n){
        ll ln = (i+1-closestCut);
        if(ln+i > n) break;
        if(ha.get(i-ln+1,i+1) == rha.get(i,ln+i)){
            canCutPref[i] = true;
            closestCut = i;
        }
    }

    vector<bool> canCutSuf(n);
    canCutSuf[n-1] = true;
    closestCut = n-1;
    for(int i=n-2;i>=0;i--){
        ll ln = (closestCut-i+1);
        if(i+1-ln < 0) break;
        if(rha.get(i,ln+i) == ha.get(i+1-ln,i+1)){
            canCutSuf[i] = true;
            closestCut = i;
        }
    }
```

```cpp

    rep(i,0,n-m+1){
        // cout<<i<<endl;
        // cout<< canCutPref[i] << " " << canCutSuf[i+m-1] << " " << (ha.get(i,i+m) == bhs) << " " <<
        if(canCutPref[i] && canCutSuf[i+m-1] && (ha.get(i,i+m) == bhs || rha.get(i,i+m) == bhs)){
            cout<<"possible"<<endl;
            _Exit(0);
        }
    }

    // reverse(all(b1));
    // if(solve(b1)){
    //     cout<<"possible"<<endl;
    //     return 0;
    // };
    cout<<"impossible"<<endl;

    _Exit(0);
}

// BABACABACABA
// 111010101000
```

Fredrik Ekholm
(Other solutions include top-down DP, and some optimized quadratic)

I - Polkagris


Problem: Problem I
Changes in Scores for Problem: Problem I
100p
1-99p
0p with try
Number of Changes
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
2025-02-01
2025-02-01
Time

J - Building a Castle

First, realize that this is basically “find 4 non-intersecting increasing subsequences of maximum length”. The K constraint doesn't really change the solution.

Or does it?

J - Building a Castle

Longest subsequence which is a union of K increasing subsequences

By **Halzion**, history, 3 years ago,

I was reading **mango_lassi**'s blog and in one of the paper referred in his blog, I found a $O(N^2)$ algorithm for constructing a longest k-increasing subsequence (a subsequence of maximum length which is a union of k increasing subsequences). While implementing it, **ko_osaga** told me about the problem E in "AMPPZ-2015 MIPT-2015 ACM-ICPC Workshop Round 1" (id #6275 in opentrain) where the problem asks to construct such subsequence with constraint $N \leq 2 \cdot 10^5$ and $K \leq 20$.

The official solution runs in $O(N \cdot K \cdot (K + \log(N)))$. The first half seems to be doing the RSK mapping, but I couldn't figure out why the second half works. It seems to be unrolling the mapping from the back, and maintaining K intervals for each row in the tableau. And the editorial only explains the first half. Can someone please explain why it works?

Official Solution

Well-known in china (RSK)

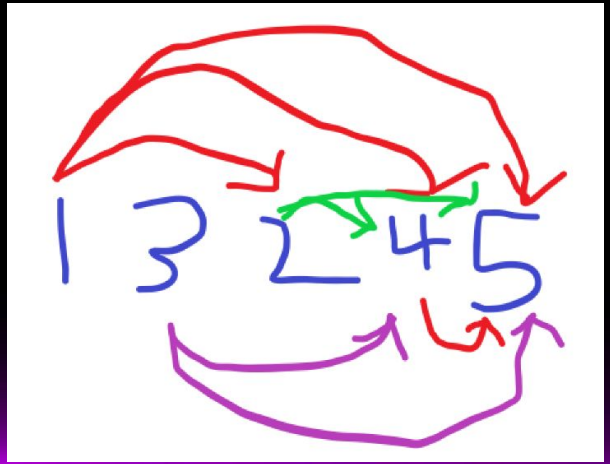
J - Building a Castle

N^5: do a DP, keep track of the heights of each tower.

$N^4\log(N)$: optimize previous DP. Use the same ideas to take normal longest increasing subsequence (LIS) from $O(N^2)$ to $O(N\log(N))$. Alternatively, writing a tight bottom-up is approximately as fast..

J - Building a Castle

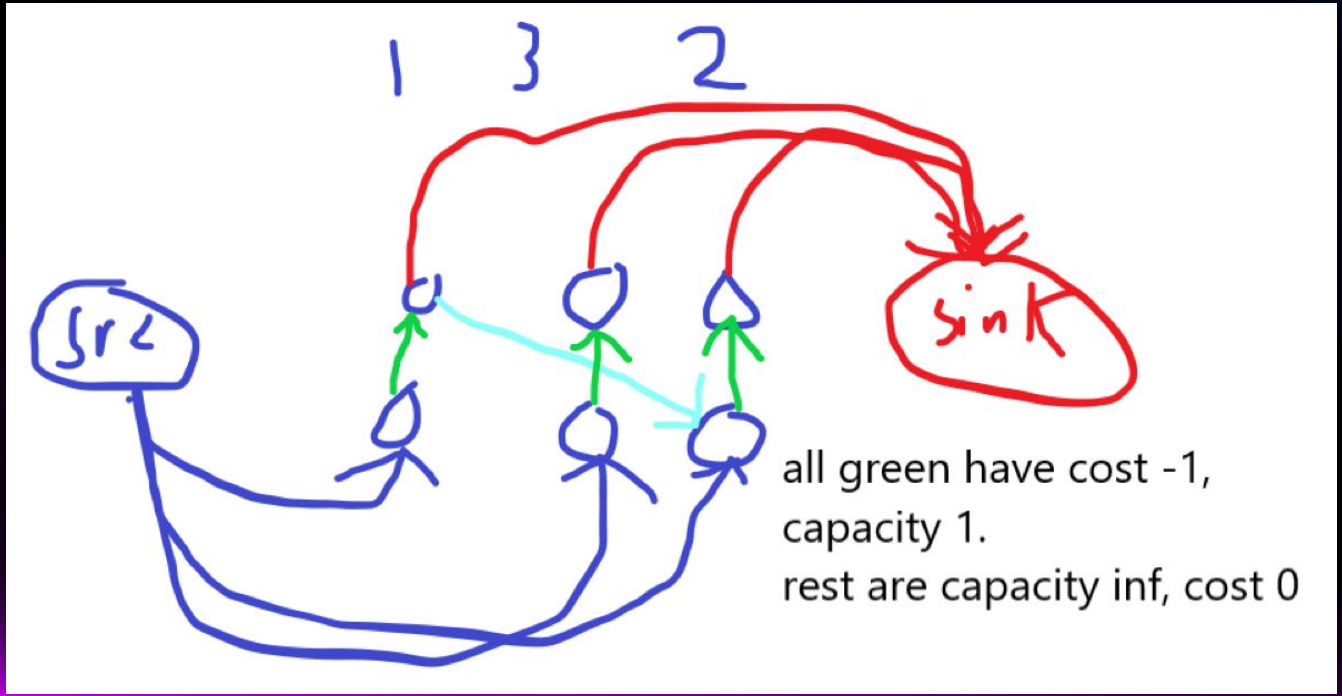
Fundamentally, DP is difficult to optimize further. Let's switch perspective. Normal LIS can be thought of as longest path, where we have a directed edge from i to j if j>i and h[j]>h[i].



J - Building a Castle

We will generalize this idea. Let's use flow. If we think a little, we can realize that letting each K-increasing subsequence be one flow, by finding min-cost max flow, we get our answer.

J - Building a Castle



J - Building a Castle

There are $O(N^2)$ edges. We only need to find 4 flow, so let's use an augmenting path-based algorithm. Two main ways:

Find each augmenting path using shortest path with negative weights

SPFA: $O(N^3)$, may perform like $O(N^2)$ in practice

Bellman-ford: $O(N^2)$

Precompute potentials, then find each augmenting pathin O(Elog(E)):

Bellman ford: $O(N^3)$, $O(N^2)$ if you introduce early breaks

SPFA: probably fast enough for $O(N^2)$

Shortest path DP: definitely $O(N^2)$

J - Building a Castle

How do we optimize further? Our graph may have $O(N^2)$ edges, so we can't consider them explicitly. What do we need to perform min cost max flow? Repeatedly find shortest path, where some edges are flipped.

If no edges are flipped, they have a simple form, so we can use a segment tree as in normal LIS.

J - Building a Castle

From here, we could probably solve the problem by carefully implementing our own min cost max flow with segment tree+potentials. The judge(s) have not bothered with this.

J - Building a Castle

Well-known fact: in a random permutation, expected length of LIS is O(sqrt(N)). We assume worst-case of K=1. We can guess that 4 LIS length is ~4sqrt(N). Thus, we can infer that there will be at most that many flipped edges.

If we only go left-to right in the DAG, we can do this using segment tree in O(Nlog(N)). We can also go right-to-left using flipped edges in O(sqrt(N)) (remember, not many flipped edges).

J - Building a Castle

Find the shortest path in each direction alternatingly until no more distances are updated. If the order of edge relaxations right-to-left are wrong, it will run in ~O(4*4sqrt(N)Nlog(N))

If the order of edge relaxations is proper, it will take about 9 back-and-forths. ~O(4*9*Nlog(N)). Works because random permutation

J - Building a Castle